UNIVERSIDAD AUTÓNOMA DE MADRID

DEPARTAMENTO DE INFORMÁTICA

Fundamentos de Bases de Datos

Práctica - 2

Autor: Ioel Nieto y David Chacón

Curso 2026-2027

Índice

1. Introducción **2**

2. Submenú Products **2**
2.1. Stock 2
2.2. Find 2

3. Submenú Orders **2**
3.1. Open 2
3.2. Range 3
3.3. Detail 3

4. Submenú Customers **3**
4.1. Find 3
4.2. List Products 4
4.3. Balance 4

1. Introducción

Este documento presenta la memoria de la Práctica 2. El programa desarrollado en C utiliza la librería ODBC para operar con la base de datos `classicmodels`. Para proteger el sistema contra ataques de inyección SQL, el programa procesa de forma segura todos los datos introducidos por el usuario. El código utiliza sentencias preparadas con las funciones `SQLPrepare` y `SQLBindParameter` para ejecutar las consultas parametrizadas.

2. Submenú Products

Este submenú ofrece opciones para la consulta y búsqueda del catálogo de productos.

2.1. Stock

Lógica implementada: El programa solicita al usuario un identificador de producto (`productcode`) por teclado. La cadena obtenida se limpia de los posibles retornos de carro generados por `fgets` y se enlaza como parámetro (`SQL_PARAM_INPUT`) a la consulta preparada. Tras la ejecución (`SQLExecute`), la variable enlazada recoge la cantidad en stock y el programa itera con `SQLFetch` para imprimirla en pantalla.

Comando SQL:

```
SELECT quantityinstock FROM products WHERE productcode = ?
```

2.2. Find

Lógica implementada: El usuario proporciona una cadena de texto para buscar productos por nombre. Para permitir una búsqueda de coincidencia parcial en el interior de los nombres de los productos, la cadena capturada se concatena con el carácter comodín `%` por delante y por detrás utilizando `sprintf` antes de enlazarla. Por requerimiento, los resultados (código y nombre) son extraídos y mostrados en orden ascendente según el código de producto.

Comando SQL:

```
SELECT productcode, productname FROM products WHERE productname LIKE ?
   ORDER BY productcode
```

3. Submenú Orders

Esta sección aborda la lógica de seguimiento, filtrado por fechas y desglose de los pedidos.

3.1. Open

Lógica implementada: Al no requerir intervención del usuario, el *statement* se prepara de forma estática para buscar únicamente aquellos pedidos cuya fecha de envío (`shippeddate`) permanezca vacía o nula. Posteriormente, el programa recorre los registros resultantes e imprime la lista ordenada de identificadores de pedido generados.

Comando SQL:

```
SELECT ordernumber FROM orders WHERE shippeddate IS NULL ORDER BY
    ordernumber
```

3.2. Range

Lógica implementada: Se extraen dos fechas provistas por el usuario siguiendo un formato fijo (`YYYY-MM-DD - YYYY-MM-DD`) mediante la función `sscanf`. Ambas se enlazan como parámetros secuenciales correspondientes a los límites inferior (`>=`) y superior (`<=`) de la fecha de pedido (`orderdate`). La robustez en la visualización se consigue evaluando la variable indicadora devuelta (`ind_shipped`); si resulta ser `SQL_NULL_DATA`, la cadena vacía se reemplaza manualmente con el texto "NULL.ªntes de imprimirla por pantalla.

Comando SQL:

```
SELECT ordernumber, orderdate, shippeddate FROM orders WHERE orderdate
    >= ? AND orderdate <= ? ORDER BY ordernumber
```

3.3. Detail

Lógica implementada: Se solicita el `ordernumber` y se ejecutan dos consultas preparadas de forma consecutiva bajo la misma conexión. La primera combina (`JOIN`) las tablas de pedidos y detalles de pedido aplicando un `SUM` para calcular el importe total del mismo agrupando por fecha y estado; tras imprimir su resultado y cerrar el cursor localmente con `SQLCloseCursor`, se prepara y ejecuta una segunda consulta independiente filtrando la tabla `orderdetails` para extraer el desglose unitario y cantidad de productos ordenados por el índice de línea de detalle (`orderlinenumber`).

Comandos SQL:

```
-- Consulta 1: Datos globales del pedido
SELECT o.orderdate, o.status, SUM(od.quantityordered * od.priceeach)
    FROM orders o JOIN orderdetails od ON o.ordernumber = od.ordernumber
    WHERE o.ordernumber = ? GROUP BY o.orderdate, o.status

-- Consulta 2: Desglose de lineas del pedido
SELECT productcode, quantityordered, priceeach FROM orderdetails WHERE
    ordernumber = ? ORDER BY orderlinenumber
```

4. Submenú Customers

El menú de clientes administra la búsqueda de cuentas y sus balances financieros.

4.1. Find

Lógica implementada: El programa recoge un fragmento del nombre o del apellido de contacto. A este fragmento se le anexan los marcadores `%` pertinentes, y se asocia idénticamente tanto al primer como al segundo parámetro del comando SQL (correspondientes a `contactfirstname` y `contactlastname`) evaluando ambas columnas en un `OR`.

Comando SQL:

```
SELECT customernumber, customername, contactfirstname, contactlastname
    FROM customers WHERE contactfirstname LIKE ? OR contactlastname LIKE
    ? ORDER BY customernumber
```

4.2. List Products

Lógica implementada: Dado un código de cliente, es necesario recorrer e interconectar `orders`, `orderdetails` y `products` para recopilar todo su historial. Para cumplir el requisito de no duplicar líneas de productos comprados en múltiples pedidos separados, la consulta utiliza la agregación `SUM` sobre la columna `quantityordered` y una restricción `GROUP BY` anclada al nombre del producto.

Comando SQL:

```
SELECT p.productname, SUM(od.quantityordered) FROM products p JOIN
    orderdetails od ON p.productcode = od.productcode JOIN orders o ON od
    .ordernumber = o.ordernumber WHERE o.customernumber = ? GROUP BY p.
    productcode, p.productname ORDER BY p.productcode
```

4.3. Balance

Lógica implementada: El cálculo del saldo del cliente demanda sustraer la suma de lo facturado a la suma de sus pagos registrados. Debido al riesgo de valores nulos al interconectar clientes sin registros temporales en algunas de ambas tablas, se ha delegado toda la computación a la base de datos mediante subconsultas escalares en la cláusula `SELECT` anidadas con `COALESCE` para forzar la conversión de nulos a cero antes de aplicar la diferencia. Se enlaza el número de cliente dos veces para las dos subconsultas implicadas.

Comando SQL:

```
SELECT (SELECT COALESCE(SUM(amount), 0) FROM payments WHERE
    customernumber = ?) - (SELECT COALESCE(SUM(od.quantityordered * od.
    priceeach), 0) FROM orders o JOIN orderdetails od ON o.ordernumber =
    od.ordernumber WHERE o.customernumber = ?) AS balance
```